

Universidad de Las Palmas de Gran Canaria

Grado en Ciencia e Ingeniería de Datos

Big Data: Search Engine Project

Stage 1: Building the Data Layer

Course: Big Data
Group: PIE data
Repository: https://github.com/PIE-data/stage_1

Group members

Name	Student ID
Marcela Nieto-Kuczyńska	TODO
Andrea Patruno	TODO
Ivan Colella	TODO
Manuel Muñoz Roca	TODO

Academic Year 2026/2027

Contents

2	Introduction and Objectives	1
3	System Architecture	1
3.1	Datalake	1
3.2	Datamarts	2
3.3	Control Layer	2
4	Design Decisions	2
4.1	One Specification, One Oracle	3
4.2	Programming Languages: Python, Node.js, and Go	3
4.3	Tokenization and Index Granularity	3
4.4	Corpus and Download	4
4.5	Datalake Layouts	4
4.6	Inverted Index Backends	4
4.7	Metadata Datamart	5
4.8	Control Layer	5
4.9	How We Measure	6
5	Benchmarks and Results	6
5.1	Dataset, Methods, and Measures	6
5.2	Datalake Results	7
5.3	Inverted-index Results	11
5.4	Supplementary Metadata Datamart Results	15
5.5	Results Compared with Expectations	16
5.6	Threats to Validity	16
6	Conclusions and Future Improvements	17

2 Introduction and Objectives

The overall goal of the Big Data course project is to design and implement a simple search engine from scratch. Stage 1 focuses on the data layer: the part of the system responsible for acquiring documents, transforming them into consistent representations, storing them efficiently, and tracking the state of the pipeline.

The dataset is based on Project Gutenberg. Each book is downloaded as plain text and split using the Project Gutenberg start and end markers into a header and a cleaned body. The header is used to extract structured metadata, while the body is tokenized to build a word-level inverted index. The footer is not required by the later search pipeline and is discarded.

The objective of this stage is not only to build a working implementation, but also to compare alternative physical storage designs under controlled conditions. The pipeline is implemented in Python, Node.js, and Go, using the same corpora, normalization rules, workloads, and output contract in every language. This allows performance differences to be attributed to implementation and storage choices rather than to differences in functionality.

The result of Stage 1 is a reliable and reproducible storage architecture that separates unstructured book content from structured, queryable data. A control layer makes the pipeline restartable and idempotent. This foundation will be reused in Stage 2 for ranking, phrase queries, and more advanced search functionality.

3 System Architecture

The system architecture is structured into three distinct physical and logical layers: the Datalake (raw artifact storage), the Datamarts (structured metadata and indices), and the Control Layer (state tracking and concurrency). Figure 1 illustrates the data flow from ingestion to querying.

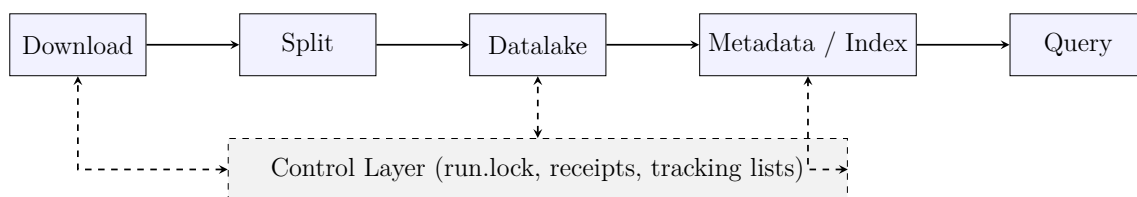


Figure 1: System pipeline and Control Layer interaction.

3.1 Datalake

The Datalake stores the split artifacts resulting from the ingestion of Gutenberg books. The system guarantees atomic writes and supports three mutually exclusive directory layouts:

- **Time-based layout:** artifacts are partitioned by UTC ingestion time: `datalake/<YYYYMMDD>/<HH>/<id>.body.txt` and the corresponding header file.

- **Book-based layout:** artifacts are grouped by book ID: `datalake/books/<id>/body.txt`, `header.txt`, and `meta.json`.
- **Batch/prefix layout (CLI name hash):** artifacts are sharded using the zero-padded six-digit decimal book ID, with `id6[0:2]/id6[2:4]`. Despite the historical CLI name, this is a digit-prefix partitioning scheme, not a hash function.

3.2 Datamarts

The Datamarts transform the raw datalake files into structured queryable formats, separating metadata from the inverted index.

Metadata Store. The parsed header fields are stored in a centralized SQLite database at `datamarts/metadata.db`. The database uses Write-Ahead Logging (WAL) and stores book identity, parsed title/author/language fields, workspace-relative paths, byte counts, content hashes, and ingestion timestamps.

Inverted Index Backends. The positional inverted index stores document IDs, term frequencies, and token positions. Three interchangeable physical backends are implemented:

- **JSON:** one monolithic file at `datamarts/inverted_index.json`.
- **Folder:** one file per term under `datamarts/inverted_index/<BUCKET>/<safe_term>.txt`.
- **SQLite:** an embedded B-tree database at `datamarts/index.db`, using `terms` and `postings` tables.

3.3 Control Layer

The Control Layer governs execution state, restartability, and concurrency. All control artifacts live under `control/`:

- **State tracking lists:** `downloaded_books.txt` and `indexed_books.txt` record successfully completed stages, while `failed_books.txt` records failures and their reasons.
- **Concurrency lock:** `run.lock` prevents overlapping writers.
- **Ingestion receipts:** JSON records under `control/ingestion/<layout>/<id>.json` bind a book to its ingestion time and final artifact paths.
- **Reconciliation:** after a crash, the control state can be rebuilt from complete datalake artifacts, allowing the pipeline to converge to a consistent state.

4 Design Decisions

Section 3 describes *what* the data layer contains; this section explains *why*. Every decision was taken to make the comparison of Section 5 fair first and fast second: three implementations that do not provably do the same work cannot be compared at all.

4.1 One Specification, One Oracle

All three implementations are written against a single specification, `docs/SPEC.md` (version 1.1.9). It fixes the command-line interface, the tokenizer, the header-parsing rules, the datalake layouts, the index and metadata formats and the exit codes. Every CLI refuses to run when its supported version differs from `spec/SPEC_VERSION`, so a change to the specification cannot silently leave one language behind.

Equivalence is checked, not assumed. Each implementation exports its inverted index in a *canonical* form (JSON, UTF-8, no insignificant whitespace, terms in UTF-8 byte order, postings by ascending book id, positions ascending), and the SHA-256 of that export must equal a frozen value computed on 20 golden books. Continuous integration runs this *conformance gate* for every language $\times$ index backend $\times$ datalake layout: all 27 combinations produce the same hash. The metadata datamart is checked the same way during the benchmarks: the runner hashes every row that each language writes, and the three languages produce identical rows for every layout and corpus size.

Two details of the canonical form exist only because of the other languages: terms are sorted by their UTF-8 bytes, because JavaScript’s native string order (UTF-16) differs for characters outside the Basic Multilingual Plane; and Go’s JSON encoder must not escape `<`, `>` and `&`, which it does by default.

4.2 Programming Languages: Python, Node.js, and Go

Python is the reference implementation. Node.js and Go were chosen over Java because neither needs a build system to get started, and together the three cover the execution models we wanted to contrast: an interpreter (CPython), a just-in-time compiled runtime with an event loop (V8) and ahead-of-time compiled native code with lightweight threads (Go). The work was split by language on top of the shared specification, with one rule: nobody ported a component that affects the canonical hash and that they had written in Python, so that each port is an independent reading of the specification rather than a translation of the reference code. The rule was not kept everywhere: parts of the Node.js JSON backend and of the Go command-line wiring and index batching were completed by the author of the Python reference late in the project. Section 5 lists this among the threats to validity.

4.3 Tokenization and Index Granularity

Text is normalised with NFKC, lower-cased independently of the locale and folded to ASCII where a mapping exists. Tokens are maximal runs of letters, digits and apostrophes; tokens shorter than 2 or longer than 40 code points, tokens made only of digits and English stop words are discarded. Positions are assigned *before* filtering, so a gap between two positions marks a removed stop word.

The index is word-level: for every term and book it stores the term frequency and the positions. This costs more space than a list of book ids, but it is what phrase queries and ranking functions such as TF-IDF or BM25 need in Stage 2.

We deliberately do **not** stem. Stemming libraries in the three languages do not produce byte-identical output, which would break the conformance hash, and a hand-written

stemmer in three languages was out of scope. For the same reason, regular expressions are allowed only to classify a single code point by Unicode category: regular-expression engines differ between languages precisely in the corners (Unicode classes, word boundaries) that decide where a token ends.

4.4 Corpus and Download

English books only, so that one stop-word list applies to the whole corpus. Fixed manifests of 100, 1 000 and 10 000 books (`spec/corpus/`) give a factor of ten between sizes; the datalake and index measurements use 100 and 1 000 books, the metadata measurements go up to 10 000. Every benchmark downloads from a local HTTP mirror of Project Gutenberg: against the live site we would be measuring internet latency and its rate limiter, not our code.

4.5 Datalake Layouts

The three layouts trade lookup cost against the cost of finding new books:

- **time** groups books by ingestion hour. Finding the books ingested since a given instant only reads the folders from that hour on; looking up one book by id means searching every hour folder.
- **book** gives every book its own folder. Lookup is a direct path, and the folder can hold a self-describing `meta.json`; finding new books requires checking the modification time of every book.
- **hash** places a book in a two-level folder taken from the digits of its id, padded to six (`001342` $\rightarrow$ `00/13`). Lookup is a direct path and no folder holds more than 100 sub-folders, without one folder per book.

Every file is written atomically (temporary file, flush, `fsync`, rename, `fsync` of the parent folder), so a crash leaves either the old file or the new one, never half a book. In the benchmarks the time layout is filled at 100 books per hour: with a single ingestion instant every book would land in one folder and the layout would degenerate into a flat directory, which is not the structure being evaluated.

4.6 Inverted Index Backends

The three structures hold exactly the same information and differ only in how they store it: one large file (**json**), many small files (**folder**) and an embedded B-tree (**sqlite**).

- **json**: updating the index means reading, merging and rewriting the whole file. Incremental side files are forbidden by the specification precisely so that the update experiment measures that cost honestly.
- **folder**: one file per term, in 27 buckets (first letter, or `_`). Term names are percent-encoded because raw terms collide as file names on case-insensitive file systems (NTFS, macOS) and on reserved names such as `CON`. Updating a book rewrites only the files of its terms.
- **sqlite**: postings live in a `WITHOUT ROWID` table keyed by (`term`, `book_id`), so the

postings of a term are read from the primary-key tree itself with no second lookup; document frequency is kept in a separate `terms` table because every query reads it.

SQLite was preferred to MongoDB as the third structure: it needs no external service, its drivers are mature in all three languages, it is already used for the metadata datamart, and it turns the comparison into a clean axis — one file, many files, one embedded B-tree — instead of files versus a network server.

Implementation choices that matter for the measurements. Books are indexed in batches of 500: one rewrite of the JSON file, one rewrite of each touched term file and one SQLite transaction per batch, with document frequencies refreshed once per transaction for the terms touched. The JSON file holds exactly the canonical bytes in all three languages, so the three write the same amount of data. SQLite runs with a write-ahead log, `synchronous=NORMAL` and a 256 MiB page cache. None of these choices changes the content of the index — the canonical hash is the same — but they keep the measurements about the data structures rather than about avoidable overheads in our code; they are part of the specification, so every language follows them. The SQLite driver differs between languages: Python and Node.js use the C library, while Go uses modernc.org/sqlite, a translation of SQLite into Go that avoids a C compiler but is slower.

4.7 Metadata Datamart

Header fields are parsed into one SQLite table with indexes on author, language and title, the fields the benchmark queries filter on. The parsing rules are fixed in the specification because each of them caused a divergence between languages during development: fields start only at column 0 with a known name; an indented line continues the current field, and blank lines do not end it, because in some Gutenberg files the lines of a long field are separated by empty lines; the release date is a single line, and the indented “Most recently updated” line under it is ignored; month names come from a fixed English table, never from a locale-dependent date parser; languages are mapped to ISO 639-1 codes through a shared file.

The ingestion instant of a book is recorded once, in a small receipt written when the book enters the datalake (`control/ingestion/<layout>/<id>.json`), and is the only source of the `ingested_at` field. Rebuilding the metadata therefore always gives the same record, whatever the layout and whenever it is run, and the book layout’s `meta.json` is written at ingestion time from the same record.

Comparing alternative metadata stores is optional in the brief and was not done: the metadata stays in SQLite, and its build time and query time are measured instead.

4.8 Control Layer

The pipeline state lives in plain text files. A book is recorded as downloaded only after its files and its receipt are durable, and as indexed only after the batch containing it has been committed: a crash can therefore cause a book to be processed twice, never to be lost, and processing it twice is harmless because re-indexing replaces a book’s postings instead of appending to them. `reconcile` rebuilds the control files from what is actually on disk,

and an operating-system file lock prevents two runs on the same workspace — a lock that disappears when the process dies, so a killed run can always be restarted. Each step of the control loop moves one book one stage forward, indexing pending books (smallest id first) before downloading the next one in manifest order, so the three languages process the same books in the same order.

4.9 How We Measure

Measurements are taken on two layers. The *macro* layer runs each CLI command as a separate process, timed from outside with a monotonic clock, with peak memory taken by `/usr/bin/time` on the child process — never self-reported — so every language pays for its own runtime in the same way. Each command also reports its own internal time, which excludes interpreter start-up: for operations that take a few milliseconds, such as detecting new books, start-up (70–150 ms for Python) would otherwise be most of the number. Lookup and query are measured this way in all three languages too, one call per book or per query, and the runner checks that every language returns the same paths and the same book ids. The *micro* layer times the same operations, and the metadata queries, inside the Python process with `pytest-benchmark`, with the index already in memory and a warm cache.

Every configuration is run once as warm-up and then three times (the folder backend, which takes about ten minutes per run, without warm-up); the workspace is prepared outside the timer and the operating-system page cache is dropped before every measured run. We report medians and interquartile ranges rather than means, because background activity on a laptop produces occasional runs several times slower than the median.

5 Benchmarks and Results

5.1 Dataset, Methods, and Measures

Table 1: Benchmark design used for the Stage 1 evaluation.

Dataset	Method	Measures
20-book golden corpus	Python, Node.js, and Go; common specification and canonical export	Cross-language and cross-backend conformance
1,000-book English corpus	Language $\times$ datalake layout: time, book, batch/prefix	Download/write throughput; lookup latency; incremental-processing cost; recovery behavior; storage overhead
1,000-book English corpus	Language $\times$ index backend: JSON and SQLite	Index-build time; update time; peak RSS; time-memory trade-off
100-book English corpus	Language $\times$ folder index backend	Index-build and update cost; used to establish the practical limit of the one-file-per-term design
100 / 1,000 books	SQLite metadata build and metadata-query benchmark	Build latency; query latency; query plan behavior

The primary benchmark machine was an MSI laptop with an Intel i7-1255U, 16 GB RAM (10 GiB assigned to WSL2), NVMe SSD, and Ubuntu on WSL2 with an ext4 benchmark workspace. Python 3.14, Node 22, and Go 1.26 were used. All downloads came from the local mirror.

Whole CLI commands are measured by the external macro runner so that process startup, I/O, and child-process memory are captured consistently across languages. Operations too short for meaningful process-level timing—datalake lookup and query—use an internal micro-benchmark layer. Results are reported using medians and IQR rather than means and standard deviations because the distributions are right-skewed.

5.2 Datalake Results

Figure 2 shows download and write throughput for 1,000 books. Python and Go benefit strongly from increasing the worker count from one to eight. Go improves from approximately 42 s to 9.6–11.1 s depending on the layout, while Python improves from approximately 43–49 s to 14–16 s. Node remains around 67–80 s and receives little or no benefit from eight workers. This contradicted the natural expectation that additional workers would improve all three implementations; inspection of the implementation indicates that synchronous filesystem writes serialize work on Node’s event loop.

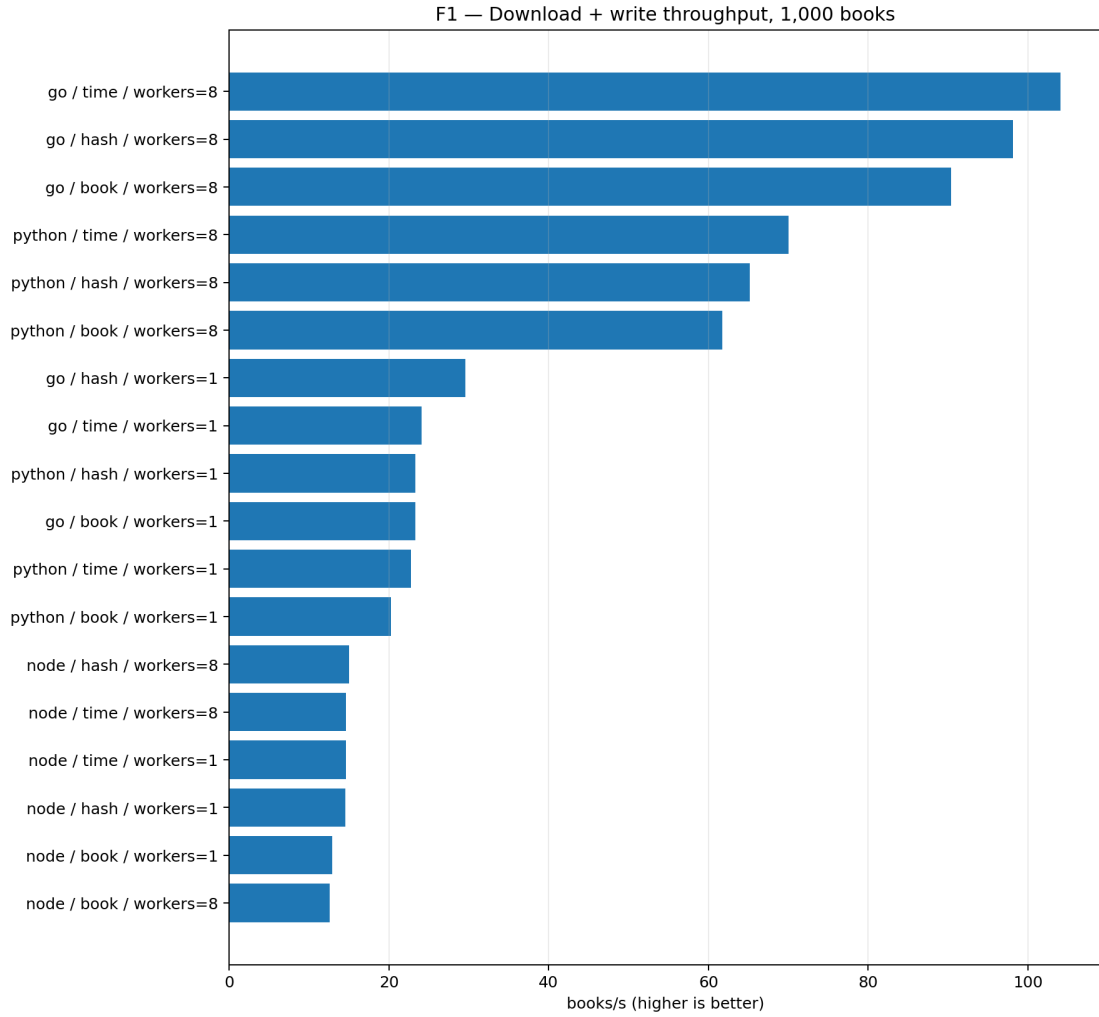


Figure 2: F1: download and write throughput for 1,000 books.

The Python micro-benchmark in Figure 3 shows that direct lookup is cheapest for the batch/prefix layout (median $75.6 \mu\text{s}$), followed by the book layout ($87.4 \mu\text{s}$). The time layout is substantially slower at $436.5 \mu\text{s}$ because lookup requires a directory walk.

Table 2 compares the three languages on the same operation through the command-line interface: `lookup` for 30 book ids drawn with a fixed seed, one call per id with a cold page cache, timed by each CLI around the command (interpreter start-up excluded). The three languages returned identical paths for every id. Go and Node.js show the same ordering as the micro-benchmark — batch/prefix fastest, time 2.6–3 times slower — while in Python the layout differences are hidden by a fixed cost of about 35 ms inside the command, largely the import of the pipeline modules, which the CLI loads only when a command needs them; the micro-benchmark above, which excludes it, is the better view of the Python layouts.

Table 2: E2 lookup through the CLI, 1 000 books: median internal time per lookup (ms), 30 lookups.

Layout	Python	Node.js	Go
time	47.3	13.6	6.5
book	41.9	5.0	3.5
batch/prefix (hash)	41.3	4.4	2.5

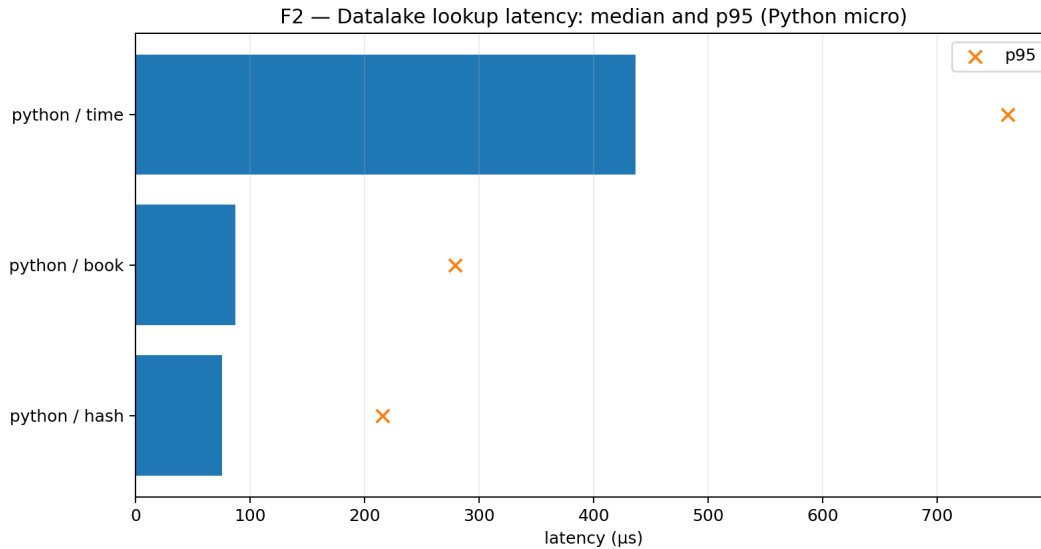


Figure 3: F2: datalake lookup latency (Python micro-benchmark).

Figure 4 compares incremental detection. The time-based layout is particularly efficient when the ingestion timestamp can narrow the scan, while the prefix and book layouts trade some scan cost for predictable direct lookup. Figure 5 evaluates recovery after a forced termination. All tested language/layout combinations recovered without losing or duplicating books, which is more important for this experiment than the small performance differences between layouts.

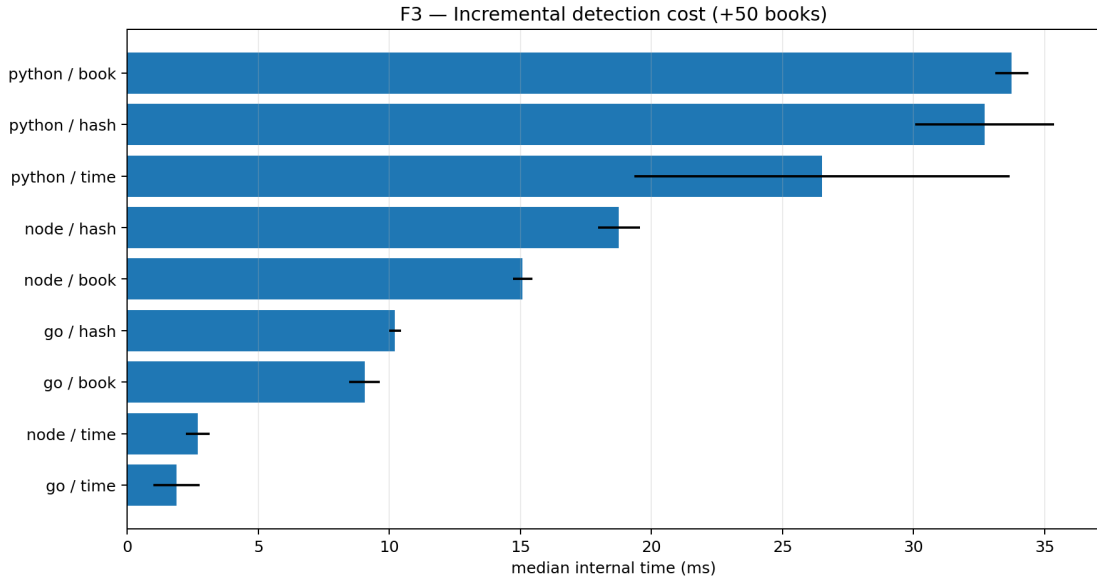


Figure 4: F3: incremental detection cost after adding 50 books.

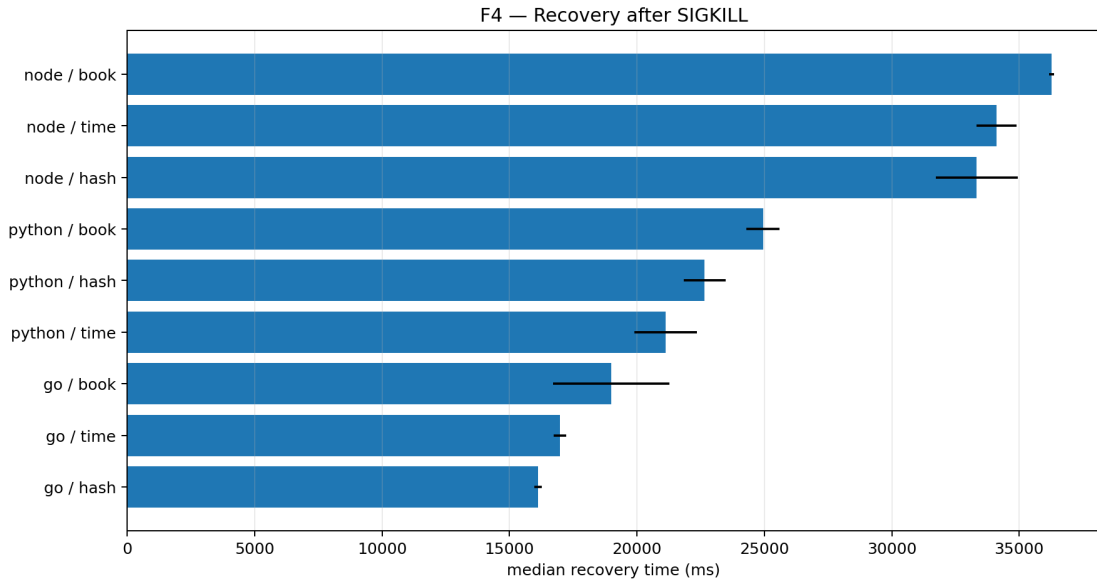


Figure 5: F4: recovery time after SIGKILL at approximately 50% progress.

Storage allocation is very similar for all three layouts. Figure 6 shows approximately 391 MiB for the time layout, 393 MiB for batch/prefix, and 399 MiB for book-based storage; the spread is below 2%. Storage footprint therefore does not decide the layout choice. Table 3 adds the other quantities the brief asks for: the number of files and directories of each layout and the control structures it needs. The book layout has one more file per book (`meta.json`) and one directory per book; the time layout has the fewest directories, one per ingestion hour. The control structures are the same for every layout: the state files (downloaded, indexed and failed books), the lock file and one ingestion receipt per book.

Table 3: Storage overhead of the datalake layouts, 1 000 books.

Layout	Files	Directories	Allocated	Control structures
time	2 000	11	390.9 MB	state files, lock, 1 000 receipts
book	3 000	1 001	399.0 MB	state files, lock, 1 000 receipts
batch/prefix (hash)	2 000	563	393.1 MB	state files, lock, 1 000 receipts

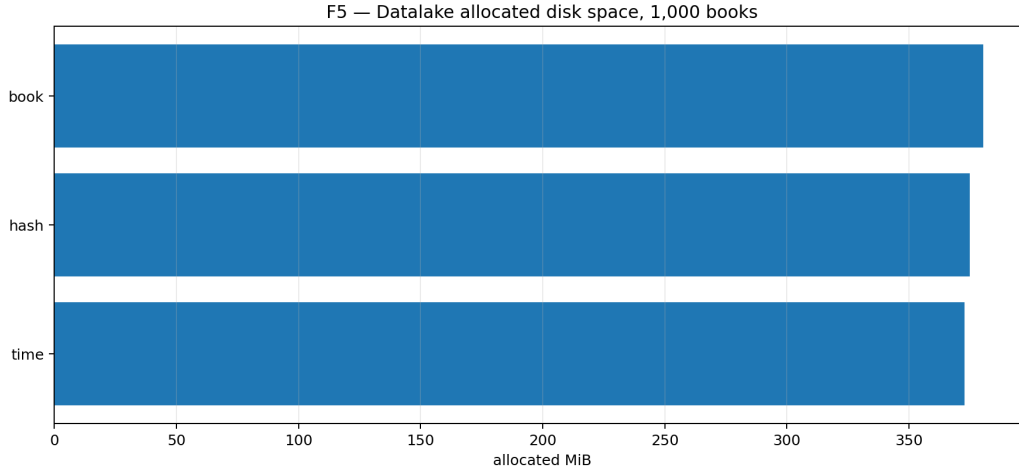


Figure 6: F5: allocated datalake storage for 1,000 books.

Selected datalake layout. We select the **batch/prefix layout** (named **hash** in the CLI) as the default. The time layout is attractive for incremental scans and slightly faster in some throughput runs, but it makes direct lookup much more expensive because a book ID does not identify its path. The book layout offers good lookup but creates one directory per book and has the largest storage allocation. Batch/prefix gives the best measured lookup, competitive parallel ingestion, bounded directory fan-out, and a deterministic path derived directly from the book ID. It is therefore the strongest overall compromise.

5.3 Inverted-index Results

Figure 7 compares index construction over 1,000 books for JSON and SQLite. Go with JSON is the fastest configuration at 29.4 s. Node takes 78.8 s with JSON and 79.4 s with SQLite. Python takes 109.5 s with JSON and 118.9 s with SQLite, while Go with SQLite takes 97.5 s.

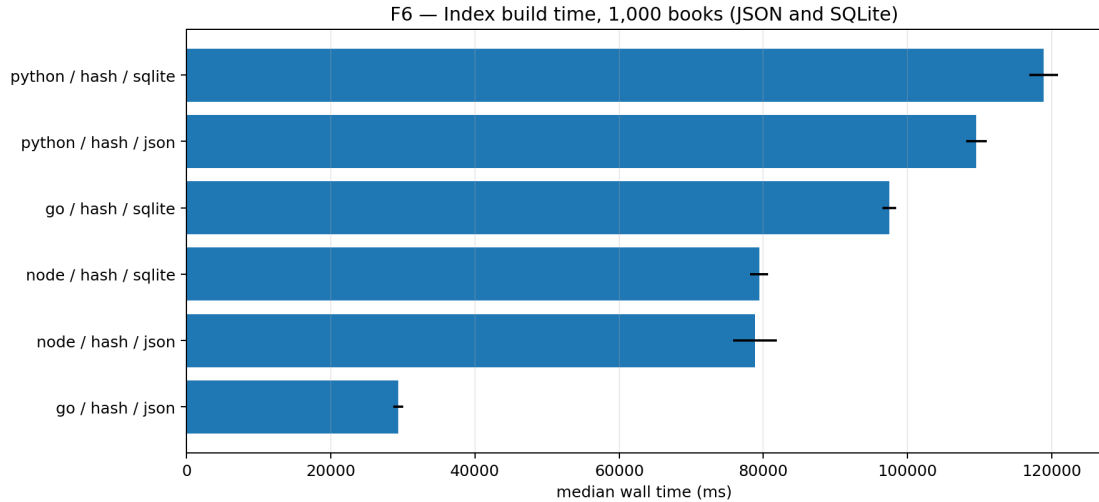


Figure 7: F6: index-build time for JSON and SQLite over 1,000 books.

Query latency is shown in Figure 8. These micro-benchmark results are currently available for Python. JSON and SQLite are almost identical for a single-term query (27.3 vs. 28.2 μ s). SQLite is somewhat faster for the AND-2 and AND-3 workloads (2.20 vs. 2.42 ms and 3.61 vs. 3.87 ms), while JSON is faster for the absent-term workload (3.6 vs. 7.9 μ s). Consequently, neither backend dominates every query type once the index is in memory.

A process that answers one query does not have the index in memory. Table 4 measures that case for the three languages: `query --mode and` through the CLI, the first 10 queries of each workload, one call per query with a cold page cache, internal time. For every one of the 40 queries the three languages and both backends returned exactly the same book ids. With JSON every query first has to read and parse the whole 278 MB index, so it costs 2.1 s in Go, 11 s in Node.js and 15 s in Python whatever the query; SQLite reads only the rows of the terms involved and answers in 10–90 ms (the Python figures include the same ≈ 35 ms of module imports as Table 2), two to three orders of magnitude faster. This is the strongest argument for SQLite as the default backend of a search engine that serves queries.

Table 4: E7 query through the CLI, 1 000 books: median internal time per query (ms), 10 queries per workload.

Backend	Language	single	AND-2	AND-3	absent
JSON	Python	15 151	14 786	14 659	14 479
JSON	Node.js	11 091	11 042	11 255	11 189
JSON	Go	2 123	2 018	2 169	2 076
SQLite	Python	45.7	54.7	89.8	23.9
SQLite	Node.js	33.8	37.9	53.0	17.5
SQLite	Go	25.4	30.8	40.5	9.6

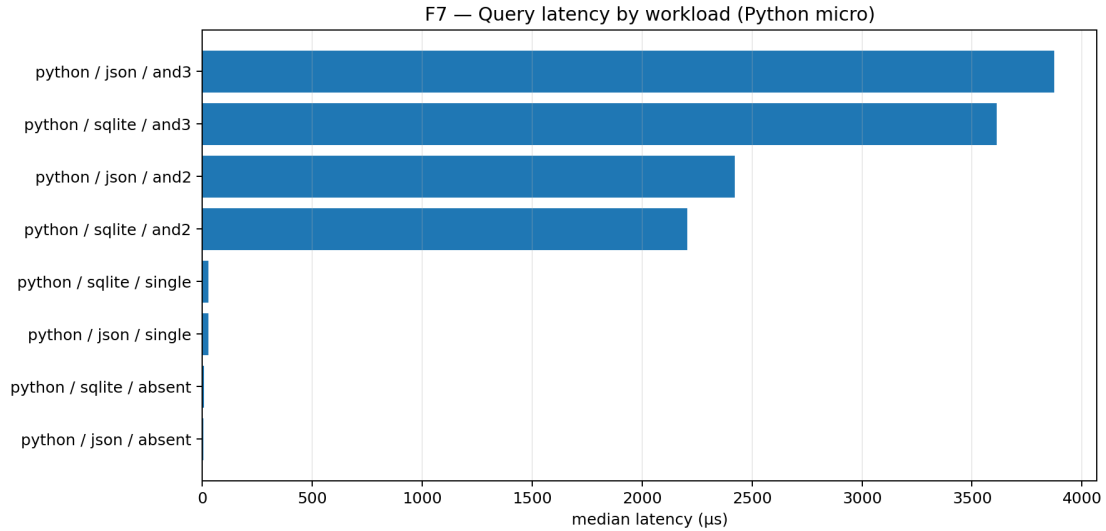


Figure 8: F7: query latency by workload (Python micro-benchmark).

For a 50-book update on top of the 1,000-book index, Figure 9 again shows different trade-offs. Go JSON is fastest at 8.6 s, followed by Node SQLite at 12.9 s. Go SQLite takes 19.3 s, Node JSON 23.0 s, and both Python configurations are about 26 s. The Node result is noteworthy because SQLite is substantially faster than JSON for the update even though their full-build times are almost equal.

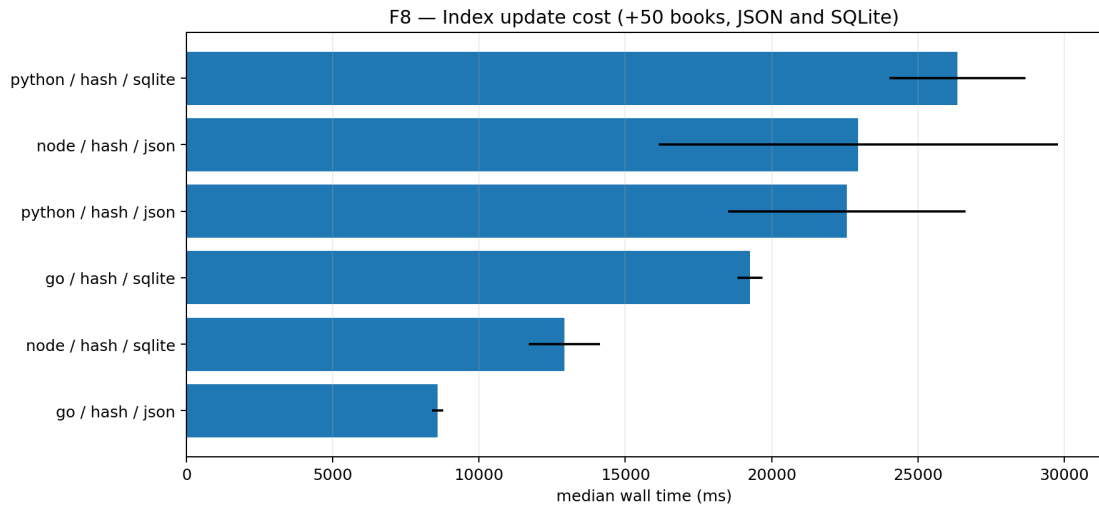


Figure 9: F8: update cost for 50 additional books.

Peak memory during full index construction is shown in Figure 10. The fastest configuration, Go JSON, is also the most memory-hungry at about 2254 MiB. Python SQLite uses the least memory at about 451 MiB. Node SQLite uses about 762 MiB, substantially less than Node JSON at about 1600 MiB.

On disk the picture is reversed: for 1 000 books the JSON index takes 278 MB and the SQLite database 500 MB, identical in size across the three languages (the JSON file is byte-for-byte the same). SQLite stores the term again in every posting row (the primary

key is `(term, book_id)`), keeps a secondary index on `book_id` and leaves free space in its B-tree pages; in exchange it can update a book without rewriting the whole index.

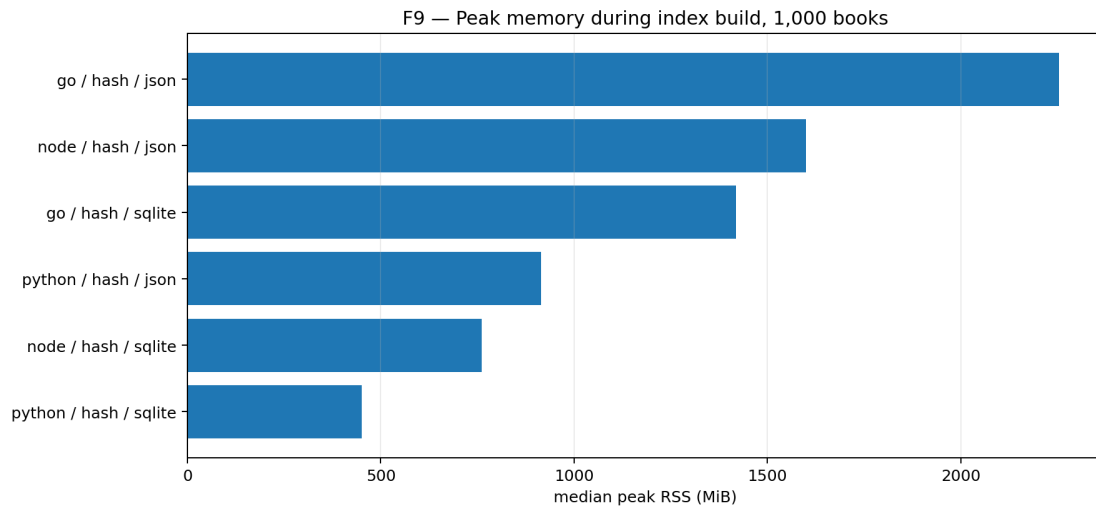


Figure 10: F9: peak RSS during index construction.

Figure 11 combines build time and memory in a Pareto analysis. The measured Pareto front contains Go JSON, Node JSON, Node SQLite, and Python SQLite. Go SQLite and Python JSON are dominated because another measured configuration is both faster and more memory efficient. This confirms that a single “fastest” result is not enough to choose a backend.

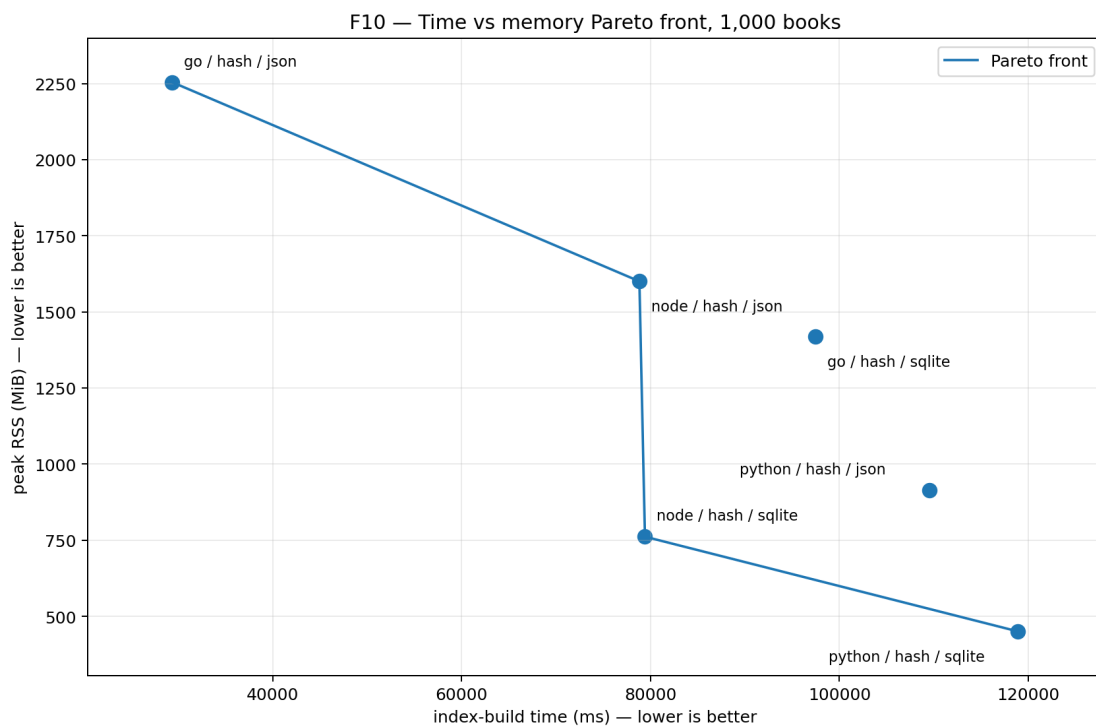


Figure 11: F10: time–memory Pareto front for 1,000-book index construction.

Scalability from 100 to 1 000 books. Table 5 repeats index build and update at 100 books. Ten times more books cost 8–15 times more build time: the postings grow with the text, while the vocabulary grows more slowly, so no configuration degrades sharply. In the update experiment the backends diverge: adding the same 50 books costs 1.9–3.2 times more with JSON on the larger index, because the whole file is read and rewritten, and 1.6–2.1 times more with SQLite, which only touches the rows of the new books and the counts of their terms; in every language the JSON ratio is the higher one. At 100 books Go with SQLite is slower than Go with JSON (12.5 vs. 3.3 s), which points to a fixed cost of the pure-Go SQLite driver rather than of the data structure.

Table 5: Index build (E6) and update with 50 books (E8) at 100 and 1 000 books: median wall time (s).

Backend	Language	E6 build			E8 +50 books		
		100	1 000	ratio	100	1 000	ratio
JSON	Python	8.9	109.5	12.3	11.0	26.4	2.4
JSON	Node.js	5.3	78.8	14.9	7.2	23.0	3.2
JSON	Go	3.3	29.4	9.0	4.5	8.6	1.9
SQLite	Python	10.3	118.9	11.5	12.4	26.3	2.1
SQLite	Node.js	6.8	79.4	11.6	7.6	12.9	1.7
SQLite	Go	12.5	97.5	7.8	12.1	19.3	1.6

Folder backend and scalability limit. The folder backend exposes the cost of the one-file-per-term design. Even at only 100 books, E6 takes 523.5 s in Go, 584.8 s in Python, and 3084.9 s in Node. Updating that index with 50 additional books takes 618.6 s, 672.3 s, and 2108.5 s, respectively. This confirms the prediction that repeated small-file creation and fsync calls would dominate the workload. The design is also expensive on disk: at 100 books the folder index holds about 102 000 term files with 20.7 MB of postings, but occupies 427 MB, because every small file takes at least one 4 KiB filesystem block — twenty times its content, against 26 MB for the JSON file and 46 MB for SQLite. Running the same backend at 1,000 or 10,000 books would have required hours and potentially around a million files, so it was not pursued.

Selected index backend. For the default Stage 1 configuration we select **SQLite**. JSON achieves the best raw build and update times in Go, but it does so with high memory consumption and requires rewriting a monolithic representation. SQLite provides more stable update semantics, a much better memory profile in Python and Node, and answers a query without loading the whole index — 10–90 ms against 2–15 s for JSON in a fresh process (Table 4). In particular, Node SQLite is almost as fast as Node JSON for a full build, uses less than half the memory, and is substantially faster for the 50-book update. Folder storage is rejected as the default because its file-per-term durability strategy is already prohibitively slow at 100 books.

5.4 Supplementary Metadata Datamart Results

The SQLite metadata datamart was also tested as a supplementary experiment. At the 1,000-book tier, metadata construction remains on the order of a few seconds across the

three languages and layouts. Batch size 500 is generally modestly faster than batch size 1; however, the difference is limited because much of the cost comes from reading bodies and computing SHA-256 hashes rather than from inserting rows.

Metadata query results also show the expected effect of indexes. Between 100 and 1,000 books, indexed lookups remain in the low-microsecond range. In contrast, the case-insensitive `LIKE prefix%` title query rises from 6.2 to 49.4 μ s because SQLite scans the table. A range formulation uses the title index and remains around 4 μ s, but it is case-sensitive and therefore is not semantically interchangeable with the original query.

5.5 Results Compared with Expectations

Several results confirm the design expectations. The folder backend was expected to be expensive because it performs one durable write per term; the benchmark confirms this very strongly. SQLite was expected to trade some raw speed for structured updates and lower memory use, which is also visible in the results.

Two observations are more informative than the initial expectations. First, increasing the worker count does not improve Node ingestion, unlike Python and Go; synchronous filesystem work serializes enough of the Node path to eliminate the expected concurrency benefit. Second, SQLite is not uniformly slower than JSON: Node SQLite is substantially faster for the incremental index update. These contradictions are therefore explained by the implementation and workload rather than treated as benchmark errors.

5.6 Threats to Validity

The results were collected on a single laptop, one NVMe SSD, one ext4 filesystem under WSL2, and one set of runtime versions. Absolute timings therefore should not be generalized to different hardware or filesystems. In particular, the folder backend is especially sensitive to filesystem behavior.

The corpus is English-only and the largest fully evaluated mandatory tier is 1,000 books. The planned 10,000-book E10/E11 scaling run was not completed, and SQLite scalability beyond the measured tier is therefore not established. The folder backend is measured only at 100 books because its one-file-per-term design was already extremely slow at that size.

The local mirror deliberately removes WAN latency and Project Gutenberg rate limiting. This improves reproducibility but means the ingestion benchmark does not represent real-world Internet download time. The number of repetitions is also modest, so medians and IQR reduce sensitivity to outliers but do not eliminate run-to-run uncertainty.

The laptop is a 15 W hybrid processor (2 performance and 8 efficiency cores) whose scheduling is decided by Windows, outside the Linux virtual machine; long runs can be throttled and a run can land on slow cores. The page cache is dropped inside Linux before every measured run, but Windows keeps its own cache of the virtual disk, so a “cold” run is only partly cold. Both effects show up as occasional outliers, which is why medians and interquartile ranges are reported.

Differences between languages are not only differences between runtimes. Go reaches SQLite through modernc.org/sqlite, a translation of the C library into Go, while Python

and Node.js call the C library itself; part of the gap between Go and Node.js on the SQLite backend is the driver. The independence of the three implementations is also weaker than planned: parts of the Node.js JSON backend and of the Go command-line wiring and batch indexing were completed by the author of the Python reference (Section 4). Their equivalence is still verified — identical canonical hashes in all 27 configurations, identical metadata rows — but some implementation choices were shared rather than made independently.

Finally, the warm-cache micro-benchmarks of lookup and query (Figures 3 and 8) exist only for Python; the cross-language comparison of the same operations goes through the CLI (Tables 2 and 4), where the internal time still contains per-command costs such as Python’s module imports. SQLite is used in place of MongoDB, so conclusions about the database-backed design apply to an embedded relational store rather than to distributed or client-server databases.

6 Conclusions and Future Improvements

Stage 1 establishes a reproducible data layer shared by Python, Node.js, and Go. The cross-language implementation uses the same data source, tokenizer rules, positional index semantics, control state, and canonical output contract, allowing the benchmark to compare storage and runtime choices rather than different functionality.

The measurements lead to two concrete default choices. For the datalake, the **batch/prefix layout** provides the best overall balance: direct lookup is cheap, parallel ingestion remains competitive, directory fan-out is bounded, and storage cost is almost identical to the other layouts. For the inverted index, **SQLite** is selected as the default because it provides a substantially better memory profile and robust update behavior while retaining competitive performance. JSON remains attractive when raw speed is the only objective, especially in Go, whereas the folder backend is not practical at larger corpus sizes under the current durability strategy.

The results also demonstrate why the three-language comparison is useful. Go with JSON delivers the fastest index construction and update, Node SQLite provides a strong time-memory compromise, and Python SQLite minimizes memory consumption. No single language/backend combination dominates every resource dimension.

Stage 2 can build ranking methods such as TF-IDF or BM25, phrase-aware queries, and richer search functionality on top of the positional index already produced in Stage 1. Future work should complete the 10,000-book scalability tier for the inverted index, run the existing Go micro-benchmarks and add Node.js ones for lookup and query, investigate asynchronous/batched Node ingestion, evaluate a less fsync-heavy folder-index update strategy, and test the selected designs on additional hardware and filesystems.